Overlay2

Volatile SD card (RAM) + a persistent store for config.json and bridge.conf: the /boot/firmware partition (default) or a USB drive. One command builds everything and turns the overlay ON.

Not yet tested on real hardware

- The logic was simulated on a computer (seed, restore, hostname/IP derivation, atomic writes, degraded mode, mosquitto.conf edit). Test on ONE master first.
- Do not clone before check_overlay2.sh shows ALL OK after the final reboot and a power-cut test passes.

1. Where things live

Item	Location
OS, programs, home, logs	SD card with overlay = RAM, never written
config.json	/boot/firmware/broker/config.json (USB mode: /mnt/persist/broker/config.json)
bridge.conf	/boot/firmware/broker/conf.d/bridge.conf (USB mode: /mnt/persist/broker/conf.d/)
mosquitto.conf	include_dir /boot/firmware/broker/conf.d (reads the store directly)
Old paths in the broker folder	Symlinks: /home/alice/broker/config.json and conf.d/bridge.conf point to the store, so apps.py, send.py, ipservice.py need no change
Master defaults	/home/alice/broker/.defaults/ (SD, read-only)
Hostname + IP	Restored at every boot from the connection name in bridge.conf (MASCAB01A=.101 ... Z=.126)

2. Install (one time, on the master)

Prerequisites: broker already installed with setup_raspberry.sh, overlay OFF, and /boot/firmware mounted read-write (check: findmnt /boot/firmware shows rw). No extra hardware is needed.

1. Check the boot partition: `findmnt /boot/firmware` (OPTIONS must contain rw)
2. Run:

```
cd /home/alice/broker/overlay2
sudo ./setup_overlay2.sh
```

3. Nothing else to type. The Pi reboots, enables the overlay automatically, and reboots once more.
4. Verify: `sudo ./check_overlay2.sh` (must end with RESULT: ALL OK)

USB mode instead: sudo ./setup_overlay2.sh --usb (drive already labelled PERSIST) or --format /dev/sda (formats it as ext4 and ERASES it). Stronger against power loss.

3. What setup_overlay2.sh does

- Writes /etc/sudoers.d/apps (alice NOPASSWD) so the :8082 overlay enable/disable button works without a password.
- Installs python3-yaml, checks /boot/firmware is writable, adds uid/gid/umask for alice to its fstab line (FAT has no owners), saves master defaults.
- Installs overlay2-prepare.service (runs before NetworkManager, mosquitto, dip, ipservice, send) and atomic writes for send and ipservice.
- Moves config.json and bridge.conf to /boot/firmware/broker, creates the symlinks.
- Changes mosquitto include_dir to /boot/firmware/broker/conf.d and log_dest to syslog; checks mosquitto can read the file and starts; restores the old file if it fails.
- Stage 2 after reboot: raspi-config nonint do_overlayfs 0, then reboot.

4. Behaviour

Situation	Result
Normal boot	Links verified, files validated, hostname and IP restored from bridge.conf
config.json or bridge.conf corrupt/missing	Restored from .bak, then from master defaults
Store missing or read-only	DEGRADED: RAM copies of master defaults, ipservice masked (no reboot loop). State in /run/overlay2.state
Symlink deleted while running	Restored at next boot, or now: sudo systemctl restart overlay2-prepare, then restart mosquitto and send
Power cut during a write	Atomic write (temp file + rename): old file stays intact

5. Tests before cloning

- Change bridge.conf from the web app, reboot: the change must remain.
- Pull the power while running, boot: check_overlay2.sh ALL OK.
- USB mode only: unplug the USB, reboot: DEGRADED, broker still runs; plug back, reboot: NORMAL.
- Press disable then enable on :8082: no password is asked and the Pi reboots each time.

6. Updating code later

Disable the overlay (button on :8082, or sudo raspi-config nonint do_overlayfs 1), wait for the reboot, copy the new code over the broker folder (do not overwrite the symlinks, .defaults or overlay2/), then enable the overlay again. The data in the store is untouched.

7. Cloning to 40 cabinets

Firmware mode: the persistent files sit on the same SD, so a clone carries the master's bridge.conf and config.json; dip.py/ipservice then set the real identity, which is saved on that cabinet's SD. USB mode: clone the SD only; each cabinet needs its OWN USB (an empty one is seeded from the master defaults).

Known limits

- Firmware mode: /boot/firmware is FAT. The guide itself warns FAT is more vulnerable to sudden power loss and has no Unix permissions; the SD is written (rarely, only when a file changes). Atomic writes and a .bak copy reduce, but do not remove, that risk. For long-lived unattended cabinets the guide prefers an ext4 partition or USB (--usb).
- A writable boot partition also reduces protection of the boot files. If boot read-only is enabled in raspi-config, setup stops and asks you to use --usb.
- The first boot of a clone briefly uses the master name/IP until the DIP switch is read.
- dip.py supports 15 addresses (.101-.115); 40 cabinets need another addressing scheme.
- sudoers uses NOPASSWD: ALL for alice, and the :8082 overlay button has no login (as in the current design).
- Do not combine with the old broker-persist helper (setup_overlay_persist.sh); remove it first. Uninstall: overlay OFF, then sudo ./remove_overlay2.sh.